Pramaan: Milestone 1

Complete technical notes: leakage audit, baseline classifiers and cross-dataset test

Personal project record. Written 6 November 2026.

What these notes are for. They are written so that, months from now, you can recover the technical side of Milestone 1 without remembering anything: why the project exists, why each technical choice was made, what exactly was run, what the results were, what they do and do not prove, and which mistakes were made and fixed. Pramaan means proof or evidence in Hindi and Sanskrit.

How to read them. Section 1 is a one-page summary. Sections 2 to 4 explain motive, terms and the step-by-step record. Sections 5 to 7 hold the results, claims and mistakes. Section 8 holds the reference code. Sections 9 to 12 cover the repository, how to explain the project, general lessons and open items.

Accuracy note. All numbers come from outputs you pasted during the work. A few figures were calculated by hand from your counts (marked 'by hand' or listed in Section 12.1). Items still to be filled in by you are listed in Section 12.

Contents

1. One-page summary 3
2. Why this project and why this milestone 4
3. Glossary of terms used 4
4. Step-by-step record: what we did and why 6
5. Consolidated results 11
6. What we can and cannot claim 12
7. Mistakes and corrections log 12
8. Reference code 13
9. Repository map and how to reproduce 15
10. Explaining the project to others 15
11. General lessons that carry to every project 16
12. Open items and what comes next 16

1. One-page summary

Project. Pramaan is an evidence-first fake-news verification system. The long-term idea: do not just output REAL or FAKE; find the claims in an article, retrieve evidence, and say whether the evidence supports, contradicts, or is insufficient for each claim.

Milestone 1 goal. Build a rigorous baseline and find out honestly what a simple text classifier actually learns. Questions: (1) how much of a high score on the ISOT dataset comes from shortcuts such as publisher tags and formatting, and (2) does the model still work on articles from other sources?

What we did.

- Set up a reproducible project: Python virtual environment, Git repository, `src/` code plus notebooks, and a written evaluation protocol (EVAL.md) before modelling.
- Audited the ISOT dataset and found strong shortcuts: the label is almost the same as the publisher.
- Built cleaning steps (B and C) that remove the shortcuts we could name.
- Trained TF-IDF + Logistic Regression and Linear SVM baselines on raw (A), cleaned (B) and cleaned-plus-extra (C) text, with 5-seed variance.
- Chose WELFake as a second dataset, found it overlaps ISOT heavily, removed the overlap, verified its label direction, and built a 22,779-article external test set.
- Ran the cross-dataset test, then AUC, error analysis and a dateline check.

Headline numbers.

Measure	Result
One-word rule ('no (Reuters)' means fake), raw ISOT, no machine learning	macro-F1 0.996
TF-IDF + LR on ISOT, raw / cleaned / cleaned + extra (5 seeds)	0.9915 / 0.9849 / 0.9843
Same models on 22,779 non-overlapping WELFake articles	macro-F1 about 0.57 (accuracy 58%)
Always predicting 'real' on that set (by hand)	accuracy 58.4%, macro-F1 about 0.37
ROC AUC on the external set (LR / Linear SVM)	0.721 / 0.7532

What it means. The classifier separates the two ISOT sources at about 98%, but that skill does not transfer to different sources. A classifier trained on one set of publishers largely learns those publishers. Judging truth needs evidence from outside the article, which is exactly what Pramaan's next milestones add.

What is not established. The cause of the drop (publisher style, topic, time and label quality all differ between the datasets), the quality of WELFake's labels, and whether the result holds for other datasets or non-linear models.

Repository state. Everything was committed; the closing step was the tag `v0.1-baseline` (confirm you pushed it). Progress estimate: about 35% of the committed scope by effort, about 20 to 25% of the part that makes the project distinctive (hour figures are estimates).

2. Why this project and why this milestone

2.1 The motive

News spreads fast through social media and messaging platforms, and telling reliable from misleading content is hard. A typical student project trains a classifier that outputs REAL or FAKE. The original project document already made the key point: a machine-learning prediction alone cannot establish whether a claim is true. That needs independent verification against credible sources.

So the design principle of Pramaan is **evidence first**: the classifier is at most one signal among several, and the final answer should come from claims checked against retrieved evidence, with an honest 'insufficient evidence' option. Milestone 1 tests the first half of that argument empirically: does a plain classifier really fail to be trustworthy, and why?

2.2 Personal goals that shaped the scope

- Goal: a good internship by the end of 4th semester, using personal projects to build skills and a CV. The plan discussed: 4 to 5 projects over time, namely the PBL matchmaking system (compulsory), Pramaan, a Hinglish RAG system with evaluation, and a fourth only if time allows.
- Time budget: about 7 hours per week, which gives roughly 140 hours across 4th semester, minus PBL and exams. This is why the scope was cut.
- Cut from the committed scope and listed as future work: multimodal (image, video, audio) detection, PostgreSQL, Docker and cloud deployment. Reason: they add length but little signal on a CV, and each could consume weeks. Deepfake detection is a separate research area.
- A CV project is judged on scoping, honest evaluation and finishing, not on how many phases are listed. Two deep projects beat five shallow ones because you can defend every decision.

2.3 Why Milestone 1 is a classifier plus an audit

- **Every later component needs a baseline and an evaluation protocol** to be compared against. A transformer or an evidence pipeline is only meaningful if it beats something simple under the same rules.
- **A linear model is interpretable.** Its coefficients show which words it relies on, so we can audit what it learned instead of guessing.
- **A rigorous negative result is a strong story.** 'High score collapsed on new sources, here is the diagnosis' shows judgement. The public datasets are known to let models score very high by using publisher artifacts, so we planned for it from day one.

2.4 Milestone 1 questions (from EVAL.md)

- How much of the performance on ISOT disappears when known shortcuts are removed?
- Does the model still work on a second dataset from different sources?

3. Glossary of terms used

Term	Plain meaning	Why it matters here
Repository, commit, tag	A repo is a Git-tracked project folder. A commit is a saved snapshot with a message. A tag is a permanent named bookmark on one commit.	The tag v0.1-baseline freezes the repo exactly as Milestone 1 ended.
Virtual environment (venv)	A private folder of Python packages for one project.	Stops projects interfering; requirements.txt recreates it.
Jupyter notebook, kernel	A document of code cells with outputs under each. The kernel is the Python engine that runs the cells.	Used for exploration. The kernel must be the project's .venv. Restart it after editing .py files.

Term	Plain meaning	Why it matters here
CSV vs XLSX	CSV is plain text, one row per line. XLSX is Excel's own format.	Raw data stays an untouched CSV; Excel can truncate long cells and reformat dates.
Label, source-level label	The class given to an article (here 1 = fake, 0 = real). Source-level means the label describes the publisher, not each claim.	ISOT labels are source-level, so a model can win by recognising the publisher.
Baseline	A deliberately simple model or rule that more complex ones must beat.	TF-IDF + linear models; the one-word Reuters rule.
Evaluation protocol	A written plan of data, splits, metrics and checks, made before seeing results.	EVAL.md. Prevents choosing a setup that flatters the model.
Train/test split, stratified, seed	Train on one part, measure on unseen data. Stratified keeps class proportions. A seed makes random choices repeatable.	80/20 stratified, seed 42 for main runs, seeds 0 to 4 for variance.
Data leakage, shortcut learning	The model gets clues in training data that will not exist in real use, so it scores high without learning the real signal.	The Reuters tag, quote style, URLs, @handles, subject column.
Deduplication, near-duplicate, fingerprint	Removing repeated rows. Near-duplicates are almost identical but not exact. A fingerprint is a short normalised snippet used to match copies.	Duplicates across train and test inflate scores; the fingerprint finds ISOT articles inside WELFake.
Regex (regular expression)	A search pattern for text, such as the characters (Reuters).	Used for cleaning and for measuring how common a pattern is per class.
Normalisation	Converting text to a consistent form so formatting differences disappear.	Removing quote characters and extra spaces.
TF-IDF	A score per word: high if it is frequent in this article, lower if it appears in many articles.	Turns text into numbers a linear model can use.
n-gram, min_df, sublinear tf	n-gram = run of n words (1 to 2 means single words and word pairs). min_df=3 ignores words in fewer than 3 articles. Sublinear tf damps very frequent words (uses 1 + log of the count).	Settings of our TfidfVectorizer.
Logistic Regression, Linear SVM, coefficient	Two linear classifiers. A coefficient is the weight a feature gets; positive pushes toward class 1 (fake).	Coefficients give the top-feature lists used for leakage inspection.
fit vs transform	fit learns something from data (the vocabulary). transform applies it.	The vectorizer is fitted on training text only; fitting on everything leaks test information.
Precision, recall, F1, macro-F1, accuracy	Precision: of items predicted fake, how many were. Recall: of truly fake items, how many were found. F1 combines them. Macro-F1 averages F1 over classes. Accuracy: share correct.	Macro-F1 is primary because the class balance changed and accuracy can hide a weak class.
Confusion matrix	A 2x2 table of correct and wrong predictions per class.	Shows which direction the errors go.
ROC AUC	The probability that a random fake article gets a higher fake score than a random real one. 0.5 is a coin flip, 1.0 is perfect.	Measures ranking ability independent of the decision threshold.
Ablation	Changing or removing one ingredient at a time to measure its effect.	A, B, C compare raw vs cleaned vs cleaned plus extra.
Standard deviation over seeds	How much a score moves when only the random split changes.	Tells us whether a small gap (B vs C) is real or noise.

Term	Plain meaning	Why it matters here
Distribution shift, confound	Test data differs from training data. A confound is when several things change together so you cannot tell which caused an effect.	ISOT to WELFake changes source, topic, time and label quality together.
Overlap between datasets	The same article appearing in two datasets.	Testing on overlapping articles measures memorisation, not generalisation.
Label noise	Some labels are wrong or inconsistent.	WELFake merges other datasets; label quality was not audited.
Crosstab, NaN, NaT	A table counting rows per combination of two columns. NaN is pandas' missing-value marker; NaT is the same for dates.	The label-direction check used a crosstab. A NaN bug caused the overlap miscount.
Majority-class baseline	Always predict the most common class.	On the external set it gives accuracy 58.4%.

4. Step-by-step record: what we did and why

4.1 Environment and repository structure

Goal. A project anyone (including you in six months) can rebuild and rerun.

What we did. Created a GitHub repo named pramaan, cloned it (on your machine at Documents\GitHub\pramaan), created a Python virtual environment, installed pandas, scikit-learn and jupyter, saved versions with `pip freeze > requirements.txt`, and used this layout:

```
pramaan/
  README.md  FINDINGS.md  EVAL.md  DATA.md  requirements.txt
  notebooks/   01_data_audit, 02_baseline, 03_welfake_audit, 04_cross_dataset
  src/         data.py  clean.py  overlap.py  baseline.py
  data/        not committed (raw/isot, raw/welfake)
```

Why these choices.

- **Virtual environment:** a private set of packages per project, so versions cannot clash and the setup can be recreated from requirements.txt.
- **Code in src/, exploration in notebooks:** reusable logic (loading, cleaning, metrics) lives in files that every notebook imports. This is why the Hinglish RAG project can reuse the metrics code later.
- **.gitignore for data/, *.csv, .env, .venv/, .ipynb_checkpoints/:** datasets are large (the WELFake file is over 100 MB on disk and GitHub rejects files above 100 MB), news text is owned by publishers, and licenses may forbid redistribution. You link to the source instead.
- **README, EVAL, DATA, FINDINGS as .md files:** Markdown is plain text that GitHub renders as formatted pages. README is the front page; EVAL the protocol; DATA the dataset sources and licenses; FINDINGS the results and limits.

Lesson. Reproducibility is a feature you build in from the start. It is also visible to whoever reads your repo.

4.2 Writing the evaluation protocol before modelling

Goal. Fix the rules of the experiment before seeing any result.

What we did. EVAL.md was created with sections for datasets, labels, splits, metrics, leakage checks and success criteria, plus a Deviations log: any change after seeing test results must be recorded there with the reason.

Why. If you choose metrics, splits and cleaning after seeing scores, you will unconsciously pick the setup that looks best. A written protocol and a deviations log keep the experiment honest.

Honest record. The protocol also had a section for your predictions of the A, B and C scores, to be written before running anything. Those were not recorded, which EVAL.md section 12 and the deviations log state plainly. For the next milestone, write predictions first.

4.3 Dataset 1: ISOT

What it is. The ISOT Fake News dataset from the University of Victoria's ISOT research group: 44,898 full-text articles, 23,481 fake and 21,417 real, with columns title, text, subject and date. Real articles come from Reuters; fake articles come from websites flagged as unreliable by fact-checking organisations (PolitiFact and Wikipedia), per the published description. Where to get it: the official ISOT page or Kaggle.

Why we chose it. Full articles rather than one-line claims; binary labels; widely used, so results are comparable; and it is a perfect dataset for an audit, because one class is a single wire service and the other is a mix of unreliable sites. Published work on it also notes that source information can correlate with the label.

License. A license is the owner's written statement of what you may do with the data. We recorded the rules: use for learning and research is usually fine but read the license; never upload the data; cite the source; if no license is stated, write 'not clearly stated'. The ISOT pages we found did not state a license clearly, so this remains a TODO for you to confirm on the official page. I am not a lawyer; ask your faculty for anything beyond personal research.

4.4 Getting the data to load (a troubleshooting record)

- **Files were Excel, not CSV.** They were named Fake.csv.xlsx and True.csv.xlsx (Windows hides extensions, so 'Fake.csv' can really be 'Fake.csv.csv' or an .xlsx). The loader looked for Fake.csv and raised FileNotFoundError. Fix: use the original CSVs, or read_excel with openpyxl. The row counts matched the published ones, so no data was damaged. Lesson: raw data must stay untouched; compare row counts to the published numbers after loading.
- **Debugging method used:** print the real folder contents and compare against what the code expects, instead of guessing. The traceback's last line names the exact missing path.
- **Folder-name case:** your folders were Raw/ISOT while the code uses lowercase raw/isot. Windows ignores case; Mac, Linux and Colab do not, so lowercase names are the safe convention.
- **Dates:** all 21,417 real dates became NaT at first because pandas inferred one format from the first rows and failed on the rest. Real dates look like 'December 31, 2017 ' (with a trailing space); fake dates come in several formats. Fix: keep a date_raw column, strip spaces, and parse with format='mixed'. Afterwards 0 real and 10 fake dates were unparsed; those 10 are malformed scrapes (a URL or headline sits in the date field).
- **VS Code kernel picker missing:** usually the Jupyter or Python extension is not installed, or the file opened as raw text. Fix: install the extensions, or run 'jupyter notebook' in the browser from the activated venv.

4.5 The data audit (notebook 01)

Goal. Find out whether the label is separable by anything other than truthfulness, before training.

Method. Count and compare per class: missing values, duplicates, text length, date ranges, the subject column, and the first 200 characters of ten articles read by eye. Reading raw text is what revealed the biggest shortcut.

Check (ISOT)	Real	Fake	Reading
Text contains '(Reuters)'	99.2%	0.04%	A single token gives away the label
Text contains curly quotes	50.5%	0.0%	Formatting differs; fake text shows 'couldn t' (apostrophe lost)

Check (ISOT)	Real	Fake	Reading
Texts under 10 words	1	866	Many fake rows are title-only or URL-only scrapes
Exact duplicate rows (title + text)	220	5,573	Duplicates can sit in both train and test
subject values	politicsNews, worldnews	six other values	No overlap: subject alone predicts the label
Text appearing with both labels	0	0	No contradictory duplicates

- **One-word rule baseline:** predict 'fake' when the text has no '(Reuters)'. Macro-F1 = 0.996 on the whole raw dataset, with zero machine learning. This became the bar any raw-text model must beat before we can say it learned more than the tag.
- **Dates and time:** real dates run 2016-01-13 to 2017-12-31; fake dates 2015-03-31 to 2018-02-19. After cleaning, 2015 holds only fake articles (about 1,574) and real articles start in 2016-01. The class mix flips over time (2016 Q1: 1,161 real vs 2,644 fake; 2017 Q4: 8,647 real vs 1,093 fake). Even the date format depends on the class.
- **Length:** mean words 385.6 real vs 423.2 fake; medians 359 vs 363; some texts have 0 words (an empty string is not 'missing', so isna() did not flag them).

Conclusion. In ISOT the label is almost the same thing as the publisher, and many different shortcuts separate the classes. A model scoring 99% here could be a publisher detector, not a truth detector.

4.6 Cleaning: pipelines B and C

Goal. Remove the shortcuts we can name, so we can measure how much of the score depended on them.

Pipeline B (function clean_isot):

- Drop texts with fewer than 10 words (the title-only and URL-only scrapes, 866 of them fake).
- Drop exact duplicates on (title, text) **before any train/test split**, so no article is both memorised and tested.
- Remove the Reuters dateline at the start (for example 'WASHINGTON (Reuters) -') and any '(Reuters)' tag.
- Remove all quote and apostrophe characters and normalise whitespace (fake text has no curly quotes and loses apostrophes; real text has curly quotes).
- Drop rows whose date could not be parsed (the 10 malformed scrapes; in practice it removed one extra row).

Result: 44,898 to 38,493 rows (real 21,196, fake 17,297). Real lost 221 rows (about 1%), fake lost 6,183 (about 26%), mostly duplicates and near-empty posts. The class balance moved from 48% real to about 55% real, which is why splits are stratified and macro-F1 is primary.

After B, leftover shortcuts (share of articles in each class):

Pattern	Real	Fake
The word 'Reuters' in the body (for example 'told Reuters')	23.0%	1.2%
Exchange ticker like (AMZN.O)	2.5%	0%
URL	0.2%	14.4%
'Reporting by / Editing by'	1.5%	0%
@handle	1.3%	27.3%

Pipeline C (clean_extra): B plus removal of five patterns: the word 'Reuters', exchange tickers, 'Reporting/Editing/Writing by' sign-offs, URLs, and @handles. It changed 28.3% of articles;

afterwards 'Reuters', URLs and @handles were at 0.0% overall.

Rule: measure before removing. We counted each suspect pattern per class and removed only what the numbers justified. **Freeze rule:** hunting shortcuts has diminishing returns, so cleaning was frozen once B and C scored the same.

Honest limits. Only known shortcuts are removed; reporting phrases such as 'said on' and 'on Wednesday' remain. The patterns were chosen by looking at the whole dataset (including what later becomes test data) based on class differences, not tuned to model scores. Near-duplicates inside ISOT were not examined.

4.7 The baseline models

Pipeline. Text is converted by TF-IDF (word 1 to 2 grams, min_df 3, at most 50,000 features, sublinear tf) and fed to Logistic Regression (max_iter 1000) or Linear SVM (defaults), random_state 42. The data is split 80/20 stratified with seed 42. The vectorizer is fitted on the training split only.

Why TF-IDF plus linear models.

- Fast, strong on text, and the standard baseline, so results are comparable and easy to reproduce.
- **Interpretable:** the coefficient for each word shows what the model uses. The top-30-features lists per class are the main leakage detector. A transformer would hide this.
- If a complex model does not clearly beat this baseline, especially across datasets, its added complexity is not justified.
- **No tuning in the first pass**, so the test set cannot influence settings.

Why fit on training data only. If the vectorizer learned its vocabulary from all data, test-set words would influence training: a quiet form of leakage.

Why macro-F1. It averages the F1 of each class, so the larger class cannot hide a weak one. Accuracy, per-class precision and recall, and the confusion matrix are also reported.

4.8 In-domain results (notebook 02)

First single-split run (seed 42):

Text	Macro-F1	Errors	Top 'real' features (examples)	Top 'fake' features (examples)
A raw	0.992	72 of 8,980	reuters, said, washington reuters, said on, on wednesday	getty images, https, pic twitter, hillary, image via
B cleaned	0.9834	126 of 7,699	said, said on, on wednesday, president donald	sen, even, mr, getty images, pic twitter, hillary
C cleaned + extra	0.9829	130 of 7,699	same as B	same as B

The B error count is from the corrected run, [[4199, 40], [86, 3374]]; an earlier B printout was spliced with C (see Section 7). Top features show the model reads reporting conventions and web debris, not claims.

Five-seed variance (mean +/- std): A 0.9915 +/- 0.0004; B 0.9849 +/- 0.0013; C 0.9843 +/- 0.0008.

- **A to B:** about 0.0066 lower, several times the seed spread, so it looks real but small. The error rate went from about 0.85% to 1.5%.
- **B vs C:** a gap of 0.0006 is smaller than B's spread, so they are indistinguishable. The extra patterns changed nothing measurable.
- **Caveat 1:** A to B changes several things at once (duplicates, short texts, Reuters dateline, quotes), so the gap cannot be attributed to shortcuts alone. A duplicates-only ablation would isolate that; it was not run.

- **Caveat 2:** seeds change only the ISOT training split, so the spread understates real uncertainty.

Reading. Removing every shortcut we could name cost less than one point. The classes differ in many weak ways (reporting phrases, house style, web debris), and the model uses all of them. ISOT is easy because it is really a source-identification task.

4.9 Dataset 2: WELFake

Why a second dataset. In-domain splits share publishers between train and test. Published work on unreliable-news detection found that overlapping sources between train and test act as a strong confound and that accuracy fell by more than 10% when train and test sources were separated. Only a differently sourced dataset can show whether the model generalises.

Candidates considered. WELFake (article-level, 72,134 rows, merges four existing datasets, used as an ISOT partner in other studies); UTK Fake News Detection on Kaggle (article-level, similar scale); FakeNewsNet (article-level labels but may need scraping from URLs); BharatFakeNewsKosh (Indian context, unverified); LIAR (about 12.8K one-sentence statements with six labels, a different format, better for claim-level work later). We chose WELFake for speed, with BharatFakeNewsKosh noted for Milestone 2 and the Hinglish link.

Problem 1: label direction. WELFake's documentation says 0 = fake and 1 = real, but other projects disagreed with each other. We checked the data. Probe: the share of texts containing '(Reuters)' was 60.7% under label 0 and 0.0% under label 1, so 0 is real. Proof: for the 42,671 WELFake rows that also appear in ISOT (where the true label is known), a crosstab gave 21,100 real/real and 21,570 fake/fake with just 1 disagreement. So **label_raw already follows our convention: 0 = real, 1 = fake**, and the documentation is wrong. Lesson: always verify label meaning empirically.

Problem 2: overlap with ISOT. WELFake merges other datasets, so ISOT articles may be inside it. A cross-dataset test on overlapping articles would measure memorisation. Method: build a fingerprint (characters 100 to 250 of the text after removing everything except letters and digits and lowercasing; starting at 100 skips the dateline) and match fingerprints. Result: **42,671 of 68,066 checkable rows matched (62.7%, 59.2% of all rows)**; 4,068 rows were too short to fingerprint. This catches near-identical copies, not paraphrases.

Bug found and fixed. My first helper counted missing fingerprints (NaN) as matches because NaN counts as true in Python, which over-counted overlap by exactly 4,068 rows. Fix: accept only real strings (isinstance(k, str)). How it was caught: the crosstab total (42,671) did not equal the flagged count (46,739), and the gap equalled the number of too-short rows.

External test set. Rows that were checked, did not match ISOT, have at least 10 words and are de-duplicated: **22,779 articles (13,305 real, 9,474 fake)**. Its shortcut rates differ from ISOT's:

Share of articles	ISOT real	ISOT fake	External real	External fake
'(Reuters)'	99.2%	0.04%	0.1%	0.1%
URL	0.2%	14.4%	0.2%	11.0%
@handle	1.3%	27.3%	22.2%	11.2%
curly quotes	50.5%	0.0%	90.2%	83.3%
newspaper-style dateline ('CITY - ')	n/a	n/a	21.6%	0.3%

The Reuters tag has no signal here, the @handle pattern points the opposite way, and WELFake has its own source marker (the dateline) just as ISOT has '(Reuters)'. Other WELFake facts: 558 empty titles, 783 empty texts, 9,415 duplicate texts, 1 text with both labels.

4.10 Cross-dataset experiment (notebook 04)

Setup. Train on the cleaned ISOT training split; score the same fitted model on the ISOT test split and on the WELFake external set. The external set gets the same cleaning (clean_generic, plus

clean_extra for C).

Measure	Result
ISOT test macro-F1, B / C	0.9834 / 0.9829
External macro-F1, B / C (seed 42)	0.5665 / 0.5704
External confusion matrix, B	[[4778, 8527], [1101, 8373]]
External confusion matrix, C	[[4862, 8443], [1111, 8363]]
External accuracy, B / C	57.7% / 58.1% (always 'real': 58.4%, by hand)
Reuters rule on the external set	macro-F1 0.2952 (0.996 on raw ISOT)
External macro-F1, C, LR, 5 seeds	0.5704 +/- 0.0018
External macro-F1, C, Linear SVM, 5 seeds	0.57 +/- 0.0042
External ROC AUC, C, LR / SVM	0.721 / 0.7532

- **Collapse:** about 98% in-domain falls to about 57%, near always answering 'real'. The result is the same for both models.
- **Bias toward 'fake':** 8,443 of 13,305 real external articles (about 63%) were labelled fake, while 88% of fake articles were caught.
- **AUC 0.72 to 0.75:** some signal transfers (a random fake outranks a random real about three times in four) but the decision threshold learned on ISOT does not. We did not tune the threshold on the external set, because that would leak the test labels.
- **Seed spreads are tiny** because seeds change only the ISOT training split; the external set is fixed. They do not measure uncertainty from choosing a different dataset.

Error analysis

I read 8 random real articles that the model called fake: four read like conventional news or feature reporting, two were opinion or first-person pieces, two contained embedded-tweet text ('pic. twitter'). In ISOT that tweet pattern pointed to fake, yet WELFake labels these real. Eight samples cannot show that WELFake's labels are noisy, but they justify caution.

Dateline check (hypothesis: the model learned that Reuters-style wire writing means real): real external articles with a newspaper dateline (place name, then an em dash) were predicted fake 36.9% of the time (2,876 articles) against 70.8% for those without (10,429). About 87% of false positives come from articles without the dateline, so this pattern explains little. The broader style hypothesis is **not confirmed**.

4.11 Decisions to drop or defer

- **Time-based split dropped.** The class mix shifts strongly over time (2015 has only fake articles), so a time split would mix class with time; the cross-dataset test is the stronger check.
- **No threshold tuning on external data** (it would leak test labels).
- **Transformers deferred** to a later milestone, to be evaluated cross-dataset only.
- **No further shortcut hunting** (freeze rule).

5. Consolidated results

In-domain (ISOT random 80/20 split, Logistic Regression, macro-F1, mean +/- std over 5 seeds)

ID	Text used	Macro-F1
A	Raw text as downloaded	0.9915 +/- 0.0004
B	Cleaned (clean_isot)	0.9849 +/- 0.0013
C	B plus clean_extra	0.9843 +/- 0.0008

Cross-dataset (trained on ISOT, tested on 22,779 non-overlapping WELFake articles)

Model / setting	Macro-F1	AUC	Accuracy
LR, B, seed 42	0.5665	-	57.7%
LR, C, seed 42	0.5704	0.7188	58.1%
LR, C, 5 seeds	0.5704 +/- 0.0018	0.721	-
Linear SVM, C, 5 seeds	0.57 +/- 0.0042	0.7532	-
Reuters rule (no training)	0.2952	-	-
Always 'real' (by hand)	about 0.37	0.5	58.4%

Reference points (by hand from class counts): always 'real' gives accuracy 13,305 / 22,779 = 58.4% and macro-F1 of about 0.37; random guessing gives about 0.50 macro-F1.

6. What we can and cannot claim

Can claim

- A TF-IDF classifier trained on ISOT does not transfer to a differently sourced dataset, even after removing the shortcuts we identified, and this holds for both Logistic Regression and Linear SVM across five training splits.
- ISOT's label is source-level, and a one-word rule reaches 0.996 macro-F1 on it.
- WELFake overlaps ISOT heavily (about 59% of rows) and has its own source marker.
- Some signal transfers (AUC 0.72 to 0.75) but the learned threshold does not.

Cannot claim

- The cause of the drop. Source, topic, time period and label quality are entangled.
- That WELFake's labels are reliable; their quality was not audited.
- That the A to B drop is due to shortcuts alone (it also reflects duplicates and short-text removal).
- That B beats C or the reverse (indistinguishable).
- That results generalise beyond two datasets and two linear models.
- Anything about truthfulness or fact-checking individual claims: ISOT labels describe sources, not claims.

7. Mistakes and corrections log

Recording these is part of the record. Each one produced a rule you can reuse.

What happened	Cause	Fix	Lesson
Data saved as .xlsx, loader failed	Files opened and saved in Excel; Windows hides extensions	Use original CSV or read_excel	Keep raw data untouched; show file extensions; compare row counts
Real dates all NaT	pandas inferred one format from the first rows	date_raw kept; format='mixed'	Always inspect parse failures
B and C looked identical; a wrong '130 errors' statement for B	Truncated notebook output spliced two printouts	Re-ran each in its own cell, added seed runs	Do not read truncated output; one experiment per cell
Overlap over-counted by 4,068 rows	NaN fingerprints counted as matches (NaN is truthy)	isinstance(k, str) check	Cross-check counts against independent totals
WELFake label direction	Dataset documentation was wrong	Probe plus crosstab on shared rows	Verify labels empirically

What happened	Cause	Fix	Lesson
Predictions not recorded	Skipped before running experiments	Logged in EVAL.md section 14	Write predictions first
NameError: f1_over_seeds	Function not in the saved file; notebook held an old copy of the module	Added it; restarted kernel and re-imported	Restart the kernel after editing .py files
Kernel picker not visible	Jupyter/Python extension missing or file opened as text	Install extensions or use browser Jupyter	Check tooling before debugging code

8. Reference code

These are the final versions as developed in the conversation. Your repo copy is the source of truth; if they differ, trust the repo and note why. Details such as whether you loaded CSVs or Excel copies are yours.

src/data.py

```
from pathlib import Path
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
RAW = ROOT / "data" / "raw" / "isot"
WELFAKE = ROOT / "data" / "raw" / "welfake" / "WELFake_Dataset.csv"

def load_isot():
    fake = pd.read_csv(RAW / "Fake.csv")
    fake["label"] = 1   # 1 = fake
    real = pd.read_csv(RAW / "True.csv")
    real["label"] = 0   # 0 = real
    df = pd.concat([fake, real], ignore_index=True)
    df["date_raw"] = df["date"]
    df["date"] = pd.to_datetime(df["date_raw"].str.strip(), format="mixed", errors="coerce")
    return df

def load_welfake():
    df = pd.read_csv(WELFAKE)
    df = df.drop(columns=[c for c in df.columns if str(c).startswith("Unnamed")])
    df = df.rename(columns={"label": "label_raw"})   # verified: 0 = real, 1 = fake
    df["title"] = df["title"].fillna("")
    df["text"] = df["text"].fillna("")
    return df
```

src/clean.py

```
import re

def clean_isot(df, min_words=10):
    df = df.copy()
    df["words"] = df["text"].str.split().str.len()
    df = df[df["words"] >= min_words]                          # 1. near-empty texts
    df = df.drop_duplicates(subset=["title", "text"])          # 2. exact duplicates
    df = df[df["date"].notna()]                                # 5. malformed rows
    df["text"] = df["text"].str.replace(r"^[^()]{0,60}\(Reuters\)\s*-?\s*", "", regex=True)
    df["text"] = df["text"].str.replace("(Reuters)", " ", regex=False)   # 3. Reuters tag
    df["text"] = df["text"].str.replace(r"[\"'‘’“”`]", " ", regex=True)   # 4. quotes
    df["text"] = df["text"].str.replace(r"\s+", " ", regex=True).str.strip()
    return df.reset_index(drop=True)

EXTRA_PATTERNS = {
    "reuters_word": r"Reuters",
    "ticker": r"\([A-Z]{1,5}\.[A-Z]{1,2}\)",
    "signoff": r"(Reporting|Editing|Writing) by[^.]*",
    "url": r"https?://\S+|www\.\S+",
    "handle": r"@\w+",
}

def clean_extra(df, groups=None):
    df = df.copy()
    for name, pat in EXTRA_PATTERNS.items():
        if groups is None or name in groups:
```

```
            df["text"] = df["text"].str.replace(pat, " ", regex=True)
    df["text"] = df["text"].str.replace(r"\s+", " ", regex=True).str.strip()
    return df

def clean_generic(df, min_words=10):      # same as clean_isot without the date logic
    df = df.copy()
    df["words"] = df["text"].str.split().str.len()
    df = df[df["words"] >= min_words]
    df = df.drop_duplicates(subset=["title", "text"])
    df["text"] = df["text"].str.replace(r"^[^()]{0,60}\(Reuters\)\s*-?\s*", "", regex=True)
    df["text"] = df["text"].str.replace("(Reuters)", " ", regex=False)
    df["text"] = df["text"].str.replace(r"[\"'‘’“”`]", " ", regex=True)
    df["text"] = df["text"].str.replace(r"\s+", " ", regex=True).str.strip()
    return df.reset_index(drop=True)
```

src/overlap.py

```
import re
import pandas as pd

def fingerprint(s, a=100, b=250):
    n = re.sub(r"[^a-z0-9]", "", str(s).lower())    # letters and digits only
    return n[a:b] if len(n) >= b else None          # mid-article snippet

def drop_overlap(candidate, reference, min_words=10):
    ref_keys = {k for k in reference["text"].map(fingerprint) if isinstance(k, str)}
    c = candidate.copy()
    c["key"] = c["text"].map(fingerprint)
    c = c[c["key"].notna() & ~c["key"].isin(ref_keys)]   # checked AND not in reference
    c = c[c["text"].str.split().str.len() >= min_words]
    c = c.drop_duplicates(subset=["text"])
    return c.drop(columns="key").reset_index(drop=True)
```

src/baseline.py

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, f1_score

SEED = 42

def make_model(name):
    if name == "svm":
        return LinearSVC(random_state=SEED)
    return LogisticRegression(max_iter=1000, random_state=SEED)

def _vec():
    return TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_features=50000, sublinear_tf=True)

def run_baseline(df, model="lr"):
    X_tr, X_te, y_tr, y_te = train_test_split(df["text"], df["label"], test_size=0.2,
                                              stratify=df["label"], random_state=SEED)
    vec = _vec()
    Xtr = vec.fit_transform(X_tr)         # vocabulary learned from TRAIN only
    Xte = vec.transform(X_te)
    clf = make_model(model).fit(Xtr, y_tr)
    pred = clf.predict(Xte)
    print("macro-F1:", round(f1_score(y_te, pred, average="macro"), 4))
    print(classification_report(y_te, pred, target_names=["real", "fake"]))
    print(confusion_matrix(y_te, pred))
    names = np.array(vec.get_feature_names_out())
    order = np.argsort(clf.coef_[0])      # positive coefficient = pushes toward fake
    print("top REAL:", names[order[:30]].tolist())
    print("top FAKE:", names[order[-30:]].tolist())

def f1_over_seeds(df, model="lr", seeds=(0, 1, 2, 3, 4)):
    scores = []
    for s in seeds:
        X_tr, X_te, y_tr, y_te = train_test_split(df["text"], df["label"], test_size=0.2,
                                                  stratify=df["label"], random_state=s)
        vec = _vec()
        clf = make_model(model).fit(vec.fit_transform(X_tr), y_tr)
        scores.append(f1_score(y_te, clf.predict(vec.transform(X_te)), average="macro"))
    print(round(np.mean(scores), 4), "+/-", round(np.std(scores), 4))
    return scores
```

```
def run_cross(isot_df, ext_df, model="lr"):
    X_tr, X_te, y_tr, y_te = train_test_split(isot_df["text"], isot_df["label"], test_size=0.2,
                                              stratify=isot_df["label"], random_state=SEED)
    vec = _vec()
    clf = make_model(model).fit(vec.fit_transform(X_tr), y_tr)
    for name, X, y in [("ISOT test", X_te, y_te), ("WELFake external", ext_df["text"], ext_df["label"])]:
        pred = clf.predict(vec.transform(X))
        print("===", name, "=== macro-F1:", round(f1_score(y, pred, average="macro"), 4))
        print(classification_report(y, pred, target_names=["real", "fake"]))
        print(confusion_matrix(y, pred))

def fit_isot(isot_df, model="lr", seed=SEED):
    X_tr, _, y_tr, _ = train_test_split(isot_df["text"], isot_df["label"], test_size=0.2,
                                        stratify=isot_df["label"], random_state=seed)
    vec = _vec()
    clf = make_model(model).fit(vec.fit_transform(X_tr), y_tr)
    return vec, clf
```

Key analysis snippets (run in notebooks)

```
# Reuters rule baseline (no machine learning)
rule = (~df["text"].str.contains(r"\(Reuters\)", regex=True)).astype(int)
f1_score(df["label"], rule, average="macro")            # 0.996 on raw ISOT

# Per-class rate of a suspect pattern
df["text"].str.contains(r"@\w+", regex=True).groupby(df["label"]).mean()

# WELFake label direction: compare with ISOT label on shared articles
m = wf[wf["in_isot"]].copy()
m["isot_label"] = m["key"].map(isot_label_by_key)
pd.crosstab(m["label_raw"], m["isot_label"])

# ROC AUC on the external set
roc_auc_score(cE["label"], clf.decision_function(vec.transform(cE["text"])))

# Dateline check
pat = r"^[A-Z][A-Za-z .,/]{2,40}\s—\s"
cE["dash_dateline"] = cE["text"].str.contains(pat, regex=True)
cE[cE["label"] == 0].groupby("dash_dateline")["pred"].agg(["size", "mean"])
```

9. Repository map and how to reproduce

File	Purpose
README.md	Front page: status, key results, structure, reproduction steps, limitations
FINDINGS.md	Milestone 1 results, error analysis, what cannot be concluded
EVAL.md	Protocol, audit findings, cleaning pipeline, results, checklist, deviations log, TODO list
DATA.md	Dataset sources, sizes, labels, licenses (some TODO)
src/data.py, clean.py, overlap.py, baseline.py	Loading, cleaning, overlap detection, models and metrics
notebooks/01 to 04	Data audit, baselines, WELFake audit, cross-dataset experiment
Git tag v0.1-baseline	The repo exactly as Milestone 1 ended (the closing step; confirm it was pushed)

To reproduce. Clone the repo, create and activate a venv, run pip install -r requirements.txt, place ISOT in data/raw/isot (Fake.csv, True.csv) and WELFake in data/raw/welfake (WELFake_Dataset.csv), run jupyter notebook, and run notebooks 01 to 04 in order. After editing any .py file, restart the notebook kernel.

Commit order used at closing. Findings and protocol; README and DATA; notebooks with saved outputs; then the tag.

```
git tag -a v0.1-baseline -m "Milestone 1: baselines, cross-dataset test, findings"
git push origin v0.1-baseline
```

10. Explaining the project to others

10.1 Two-minute version

I am building Pramaan, an evidence-first fake-news verification system. Before building the evidence part, I wanted to know what a normal classifier actually learns. On the ISOT dataset a TF-IDF model scores 98% macro-F1, but a one-word rule, whether the text contains '(Reuters)', scores 99.6% with no machine learning, because the labels are really about the publisher. I removed the shortcuts I could identify and the score barely moved, so I tested on a second dataset, WELFake. After removing the 59% of it that overlaps ISOT, the same model fell to about 57%, roughly the level of always answering 'real'. So the classifier learned the sources, not truth. That is why the next stage retrieves evidence for individual claims instead of classifying whole articles.

10.2 CV bullets (check every number against your notebooks first)

- Built an evaluation pipeline for fake-news classifiers (TF-IDF with linear models), including leakage audit, cross-dataset overlap detection and failure analysis.
- Showed that in-domain macro-F1 of 0.98 on ISOT falls to 0.57 on 22.8k non-overlapping WELFake articles, and traced in-domain performance to source fingerprints (a one-word rule scored 0.996).

10.3 Likely questions and honest answers

Question	Short answer
Why did the model fail on the second dataset?	I do not know the cause. Source style, topic, time and label quality all differ. I can say the drop is large, holds for two linear models, and that the model reads reporting conventions and web debris. The simple dateline hypothesis did not explain it.
How do you know the test set does not leak?	I fingerprinted articles from the middle of the text and removed the 59% of WELFake that matched ISOT. It catches near-copies, not paraphrases, so some overlap could remain.
Why macro-F1?	The class balance changed after cleaning and accuracy can hide a weak class.
Why not start with BERT?	A baseline and a protocol come first. A linear model's coefficients let me audit what it learns. Transformers come later and must be judged cross-dataset.
What would you do differently?	Write predictions before running, run a duplicates-only ablation, audit WELFake's labels, and test more than one dataset pair.
What does the result not show?	Anything about truthfulness. The labels describe sources, not claims.

11. General lessons that carry to every project

- Write the evaluation protocol and your predictions before you run anything.
- A high score is a hypothesis. Always ask what the simplest possible rule scores.
- Audit the data first: read raw examples, count patterns per class, check duplicates and overlap.
- Use at least one test set from a different source than the training data.
- Verify label meaning empirically; documentation can be wrong.
- Cross-check counts: sums, row totals and known sizes catch silent bugs.
- Run one experiment per notebook cell and do not trust truncated output.
- Measure noise (several seeds) before claiming one setup beats another.
- Record deviations, mistakes and limitations honestly; they are part of the result.
- Keep code in files, raw data untouched, and data out of Git.

12. Open items and what comes next

12.1 Items for you to fill in

- ISOT: official source link, download date, license (or 'not clearly stated'), and whether you loaded the original CSVs or the Excel copies.
- WELFake: confirm where you downloaded it and its license. The dataset's published Zenodo record is https://zenodo.org/record/4561253 (confirm it matches your download).
- Citations for both datasets, from their official pages.
- Check the hand-calculated figures against your notebooks: 58.4% always-real accuracy, macro-F1 about 0.37 for it, the 87% share of false positives, the 63% of real articles flagged, and the 59.2% overlap.

12.2 Progress

Roughly 35% of the committed scope by effort and 20 to 25% of the distinctive part. Estimates: Milestone 1 about 40 hours (done); transformer comparison 10 to 15; claim extraction, evidence retrieval and verdict 40 to 50; demo and write-up 10 to 15; total 100 to 120 hours, which is about 9 to 11 more weeks at 7 hours a week. These are estimates, not measurements.

12.3 Milestone 2 decision

- **Option A, transformer comparison:** fine-tune BERT or RoBERTa on ISOT and test cross-dataset. Needs a free GPU (Colab), about 8 to 10 hours. Likely to drop on WELFake too, which would strengthen the finding but not advance the project.
- **Option B, claim extraction and evidence retrieval:** the core of Pramaan. A fixed evidence corpus, a retrieval step and an entailment model (does the evidence support or contradict the claim), with retrieval and verdict evaluated separately. Harder and riskier, but distinctive, and the evaluation harness carries over to the Hinglish RAG project.
- My leaning was B first with a small slice of A later; this is a judgement call to settle when you scope Milestone 2.